Crypto ETL Pipeline – Full Detailed Documentation

1. Introduction

This document explains a production-style data engineering pipeline built step-by-step using Python, PostgreSQL, Docker, Airflow, and Power BI. The project evolved from a simple script into a fully orchestrated and containerized pipeline.

2. Architecture

The pipeline follows a standard ETL architecture:

API → Extract → Raw Data → Transform → Processed Data → Load → PostgreSQL → Power BI

Airflow orchestrates execution and Docker ensures portability.

3. Extract Layer

The extract step calls the CoinGecko API using the requests library. It includes retry logic, timeout handling, and validation checks.

Key responsibilities:

- Call external API - Validate response - Convert JSON to DataFrame - Store raw data as CSV

4. Transform Layer

The transform step cleans and standardizes the data.

Key transformations:

- Select relevant columns - Convert timestamps - Normalize symbols - Remove null values - Deduplicate records

5. Load Layer

The load step writes data into PostgreSQL.

Two tables are used:

- crypto_prices_current → latest snapshot - crypto_prices_history → full history

Batch tracking is implemented using batch_id and load_timestamp.

6. Logging and Error Handling

Logging is implemented using Python's logging module.

Logs are written to both console and file.

Errors are raised to ensure orchestration tools detect failures.

7. Environment Variables

Database credentials are stored in a .env file instead of hardcoding.

This improves security and portability.

8. Dockerization

Docker is used to containerize the pipeline.

Benefits:

- Consistent environment - Easy deployment - Dependency management

Key commands:

docker build -t crypto-pipeline .

docker run --env-file .env crypto-pipeline

9. Airflow Orchestration

Airflow is used to orchestrate ETL steps.

Initial DAG had one task; later split into:

extract_task → transform_task → load_task

Benefits:

- Task-level retries - Monitoring - Scheduling

10. Power BI Dashboard

Power BI connects to PostgreSQL and visualizes data.

Features:

- KPI cards - Top coins - Drill-through pages - Filters and slicers

11. Challenges Faced

- Python environment mismatch - Docker daemon issues - Airflow not working on Windows - PostgreSQL connection errors

Each issue was debugged and resolved systematically.

12. Real-World Relevance

This project mirrors real-world data engineering systems.

It demonstrates ETL design, orchestration, containerization, and analytics.

13. Future Enhancements

- Data lake structure (raw/processed) - Azure integration (ADLS, ADF, Azure SQL) - CI/CD pipelines - Monitoring and alerting

14. Interview Summary

Built an end-to-end ETL pipeline using Python, PostgreSQL, Docker, Airflow, and Power BI. Implemented modular tasks, logging, retries, and containerization for production readiness.